

Technical Guide: W: Web Pipe Device Y: Clipboard Device

Author: 13leader.net

Date: March 2026

Project Repo: <https://github.com/pjones1063/AspeQt-2k26>

Getting Started:

 **Important Note: Required Atari Drivers** To utilize the specialized devices (R:, W:, or Y:) detailed in this guide, you must load the corresponding handler into your Atari's memory. The ready-to-use drivers are located on the mydos drivers.atr disk image within the /atari folder of the AspeQt-2k26 GitHub repository. Simply mount this image to D1: and boot your Atari to load the handlers. For developers, the original 6502 assembly source code is also provided in the /atari/8-bit subdirectory.

Y: Clipboard Device (System Integration)

The Y: device provides a bi-directional bridge between the Atari 8-bit and your host computer's system clipboard (Windows, macOS, or Linux). This allows for rapid code transfer and data sharing without saving files to disk.

Usage Scenarios

- **Paste Code:** Copy BASIC listings from a website on your PC and paste them directly into the Atari editor.
- **Export Data:** Send program listings or variable data from the Atari to your PC's text editor.
- **Write to PC:** LIST "Y:" - Sends the current BASIC program to the PC clipboard.
- **Write to PC:** PRINT #1;"TEXT" - Sends specific text to the PC clipboard.
- **Read from PC:** ENTER "Y:" - Types the PC clipboard content into the Atari (like keystrokes).

- **Read from PC:** COPY "Y:", "E:" - Displays PC clipboard content on the screen.

Code Example: Reading & Writing

Writing to the PC Clipboard

Basic

```
10 REM Send a variable to the PC Clipboard
20 A$="Copied from Atari"
30 OPEN #1,8,0,"Y:"
40 PRINT #1;A$
50 CLOSE #1
```

Reading from the PC Clipboard (Ensure you have copied text on your PC first)

Basic

```
10 DIM BUFFER$(256)
20 OPEN #1,4,0,"Y:"
30 INPUT #1,BUFFER$
40 PRINT "I found this on your clipboard:"
50 PRINT BUFFER$
60 CLOSE #1
```

W: Web Pipe Device (Thin Client Gateway)

The W: device transforms the Atari 8-bit into a "Thin Client" capable of interacting with modern web services. Instead of implementing complex network stacks (TCP/IP, SSL) on the Atari, the W: device acts as a transparent proxy. It forwards standard BASIC text commands to a backend script running on your PC via the specified protocol.

1. Supported Network Protocols

The W: device understands three distinct URL schemes, each with unique networking behaviors:

A. HTTP / HTTPS (Web Requests)

Ideal for REST APIs, web scraping, and triggering backend scripts. This acts as a standard request/response batch job.

- **Read (Mode 4):** Downloads a web page or API response.
- **Write (Mode 8):** Accumulates data in AspeQt's buffer and sends it as a POST request to the URL only when CLOSE #1 is called.
- **Basic Example (HTTP GET):**

Basic

```
10 DIM R$(256)
20 OPEN #1,4,1,"W:HTTP://WTTR.IN/?FORMAT=3"
30 INPUT #1,R$
40 PRINT "WEATHER: ";R$
50 CLOSE #1
```

B. TCP (Persistent Raw Sockets)

Ideal for interactive terminals, chat clients, or live telemetry dashboards. TCP connections are bi-directional and stay open until manually closed.

- **Read/Write (Mode 12):** Allows simultaneous two-way communication.
- **Line Buffering:** When using the customized Atari assembly driver in Mode 12, the SIO network buffer is automatically flushed the instant you send a Return character (\$9B), making it perfect for instantaneous, interactive shells.

- **Basic Example (TCP Terminal):**

Basic

```
10 DIM R$(255)
20 OPEN #1,12,1,"W:TCP://10.5.5.3:9090"
30 PRINT #1;"PING"
40 INPUT #1,R$
50 PRINT "SERVER REPLIED: ";R$
60 CLOSE #1
```

C. FTP SFTP (Bulk File Transfers)

Ideal for saving or loading large files, logs, or backups to a local network server. AspeQt handles the FTP login credentials and binary protocols automatically using system-level curl commands.

- **Read (Mode 4):** Instantly downloads an entire file into AspeQt's memory pool, then feeds it to the Atari sequentially.
- **Write (Mode 8):** The Atari writes data incrementally to AspeQt's memory. The actual network upload to the FTP server is triggered only when CLOSE #1 is executed.
- **Basic Example (FTP Upload):**

Basic

```
10 OPEN #1,8,1,"W:FTP://10.5.5.3:2121/ATARI_LOG.TXT"
20 PRINT #1;"SENSOR RECORDING STARTED."
30 CLOSE #1 : REM NETWORK UPLOAD HAPPENS HERE
```

2. Multi-Channel Support (W1: - W4:)

AspeQt-2k26 supports up to 4 simultaneous connections for complex "Full Duplex" applications.

- **W1:** Primary Channel (Device \$57)
- **W2:** Secondary Channel (Device \$56)
- **W3:** Tertiary Channel (Device \$55)
- **W4:** Quaternary Channel (Device \$54)

3. On-the-Fly Mode Switching (AUX2 Flag)

By default, the W: device uses the global translation settings defined in the AspeQt Options menu. However, you can override this setting per connection using the AUX2 parameter (the 3rd number in the OPEN command).

Driver Logic (C++ Implementation) The handler determines the translation mode by examining the High Byte of the AUX register. The specific logic used in the driver is as follows:

C++

```
int aux2 = (aux >> 8) & 0xFF; // Extract High Byte
if (aux2 == 1) return true; // Force TEXT (Translate Atari $9B <-> Unix \n)
if (aux2 == 2) return false; // Force BINARY (Raw Data, No Translation)
return globalSetting; // Default (AUX2=0)
```

Usage in BASIC

- **AUX2 = 0 (Default):** Uses the "Translate EOL" checkbox setting from the AspeQt GUI.
- **AUX2 = 1 (Force Text):** Essential for reading text files or API responses (converts \n to ATASCII \$9B).
- **AUX2 = 2 (Force Binary):** Essential for downloading binary data (sprites, fonts, executables) where byte values must remain unaltered.

Basic Example (Secure SFTP Upload):

```
10 OPEN #1,8,1,"W:SFTP://admin:mypass@192.168.1.50:2222/ATARI_LOG.TXT"
20 PRINT #1;"SENSOR RECORDING STARTED."
30 CLOSE #1 : REM SECURE NETWORK UPLOAD HAPPENS HERE
```

4. Application Examples Library

The following examples demonstrate "Hybrid Computing" by offloading processing to a Python backend.

General Prerequisites For all examples below, you must have Python 3.x installed on your Host PC. Base Install: `pip install flask`

App 1: Stock Market Ticker (New) Fetches real-time stock quotes using the yfinance library. The Python script sanitizes the output (removing Unicode arrows/colors) to ensure safe display on the Atari.

- Dependencies: `pip install flask yfinance`
- Python Script: `stock_ticker.py`
- BASIC File: `STOCKS.LST`

App 2: Atari Newswire / RSS Reader (New) Turns the Atari into a teletype news reader. It fetches RSS feeds (BBC, NASA), sanitizes "smart quotes," and wraps the text to 38 characters to ensure perfect formatting on the 40-column display.

- Dependencies: `pip install flask feedparser`
- Python Script: `news_reader.py`
- BASIC File: `NEWS.LST`

App 3: Gemini AI Chatbot Turns the Atari into a client for Google's Gemini Large Language Model. You can type natural language questions ("Who is Captain Kirk?") and receive intelligent, summarized answers on the Atari screen.

- Dependencies: `pip install google-generativeai`
- Python Script: `gemini_ai.py`
- BASIC File: `GEMINI_AI.LST`

App 4: Basic Graphical Weather Station Fetches live weather data from the Open-Meteo API for a specific location (configured in Python) and displays it on the Atari using GRAPHICS 5 (4-color mode). It draws custom pixel-art icons for Sun, Clouds, Rain, or Snow.

- Dependencies: pip install requests
- Python Script: weather.py
- BASIC File: WEATHER.LST

App 5: Cloud Notepad (Persistence Demo) Demonstrates Reading and Writing to the PC's hard drive. You can type a note on the Atari, save it, turn the Atari off, and retrieve the note later. It acts as a "Cloud Drive" for text.

- Dependencies: flask (Standard)
- Python Script: cloud_note.py
- BASIC File: CLOUD_NOTE.LST

App 6: Text-to-Speech Synthesizer The "Talking Atari." Any text typed on the Atari is sent to the PC, where it is read aloud using the Linux espeak engine. This demonstrates zero-latency media triggering.

- Dependencies: flask (Python); sudo apt install espeak (Linux System)
- Python Script: tts_server.py
- BASIC File: TTS_SERVER.LST

App 7: Web News Scraper Connects to a modern news website (like CBC or BBC), scrapes the HTML, strips out ads/javascript, and delivers a plain-text headline summary to the Atari.

- Dependencies: pip install requests beautifulsoup4
- Python Script: scraper.py
- BASIC File: SCRAPER.LST

App 8: Linux Remote Execution Allows the Atari to execute shell commands on the Linux Host (e.g., `ls -la`, `whoami`). It includes a safety whitelist to prevent unauthorized access.

- Dependencies: flask (Uses standard subprocess library)
- Python Script: `exec_linux.py`
- BASIC File: `EXEC_LINUX.LST`

App 9: Windows Remote Control Similar to the Linux version but tailored for Windows Hosts. It can launch GUI applications (Calculator, Notepad) or open URLs in the default browser.

- Dependencies: flask (Uses standard subprocess library)
- Python Script: `exec_win.py`
- BASIC File: `EXEC_WIN.LST`

App 10: File Upload / Telemetry A utility to upload raw data or text files from the Atari to the PC. This can be used for saving game high scores, sensor logs, or telemetry data.

- Dependencies: flask
- Python Script: `file_upload.py`
- BASIC File: `FILE_UPLOAD.LST`

Quick Dependency Install Command To run all examples in this library, install the following packages on your Host PC: Linux / macOS / Windows:

Bash

```
pip install flask requests beautifulsoup4 google-generativeai  
yfinance feedparser psutil
```